



Naan Mudhalvan

Industrial IoT and Industry 4.0

Tektork Private Limited, Coimbatore

ThingSpeak Web Integration - Codes

Thingspeak Web Integration with Sensor Monitor

```
#include <WiFi.h>           // WiFi library for ESP32
#include <WebServer.h>       // Web server to host local dashboard
#include <HTTPClient.h>      // HTTP client for sending data to ThingSpeak

// ----- WiFi Credentials -----
const char* ssid = "SSID";   // Replace with your WiFi SSID
const char* password = "Password"; // Replace with your WiFi password

// ----- ThingSpeak Configuration -----
const char* thingspeakHost = "api.thingspeak.com"; // ThingSpeak API server
const String writeAPIKey = "CR9CQ5QZV7Q3N18T";    // Write API key to send data
const String readAPIKey  = "NJAPFN1Q80JFCS6T";    // Read API key to fetch data
const String channelID   = "3019740";             // ThingSpeak channel ID
```

```
// ----- Sensor Pins -----  
const int potPin = 36;    // Potentiometer connected to GPIO36 (ADC1_CH0)  
const int ldrPin = 39;    // LDR sensor connected to GPIO39 (ADC1_CH3)  
  
// ----- Web Server -----  
WebServer server(80);    // Create web server on port 80 (HTTP)  
  
// ----- Global Variables -----  
int potValue = 0;        // Stores potentiometer ADC value  
int ldrValue = 0;        // Stores LDR ADC value  
unsigned long lastThingSpeakUpdate = 0; // Timer for ThingSpeak uploads  
const unsigned long thingSpeakInterval = 20000; // Upload every 20 seconds
```

```
void setup() {
  Serial.begin(115200);           // Start serial monitor
  // Initialize sensor pins
  pinMode(potPin, INPUT);
  pinMode(ldrPin, INPUT);
  // Connect to WiFi
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.print("Connected to WiFi. IP address: ");
  Serial.println(WiFi.localIP());
  // ----- Setup Web Server Routes -----
  server.on("/", handleRoot);      // Route for main dashboard page
  server.on("/data", handleData);   // Route to serve local sensor data (JSON)
  server.on("/thingspeak-data", handleThingSpeakData); // Route to serve ThingSpeak data

  // Start the web server
  server.begin();
  Serial.println("Web server started");
}
```

```
void loop()
{
    // Read analog values from sensors
    potValue = analogRead(potPin);
    ldrValue = analogRead(ldrPin);

    // Send data to ThingSpeak every 20 seconds
    if (millis() - lastThingSpeakUpdate > thingSpeakInterval)
    {
        sendToThingSpeak();           // Upload function
        lastThingSpeakUpdate = millis(); // Reset timer
    }

    // Handle any incoming web server requests
    server.handleClient();

    delay(100); // Small delay for stability
}
```

```
void sendToThingSpeak() {
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;
        http.begin("http://api.thingspeak.com/update");
        http.addHeader("Content-Type", "application/x-www-form-urlencoded");

        // Prepare data: field1 = potentiometer, field2 = LDR
        String postData = "api_key=" + writeAPIKey +
            "&field1=" + String(potValue) +
            "&field2=" + String(ldrValue);

        // Send POST request
        int httpResponseCode = http.POST(postData);

        if (httpResponseCode > 0) {
            String response = http.getString();
            Serial.println("ThingSpeak Response: " + response);
        } else {
            Serial.println("Error sending to ThingSpeak: " + String(httpResponseCode));
        }
        http.end(); // End HTTP session
    }
}
```

```
void handleRoot() {
    // HTML for the ESP32 dashboard (with inline CSS + JS)
    // Displays real-time sensor readings and progress bars
    String html = "<!DOCTYPE html>";
    html += "<html lang=\"en\">";
    html += "<head>";
    html += "<meta charset=\"UTF-8\">";
    html += "<meta name=\"viewport\" content=\"width=device-width, initial-scale=1.0\">";
    html += "<title>ESP32 Sensor Dashboard</title>";

    // ----- CSS Styling for UI -----
    html += "<style>";
    html += "body { font-family: 'Segoe UI', Tahoma, sans-serif; background: linear-
gradient(135deg,#667eea,#764ba2); min-height: 100vh; padding:20px; }";
    html += ".container { max-width:1200px; margin:0 auto; background:white; border-radius:20px;
padding:30px; box-shadow:0 20px 40px rgba(0,0,0,0.1);} ";
    html += ".header { text-align:center; margin-bottom:40px;} ";
    html += ".header h1 { font-size:2.5em; font-weight:300;} ";
    html += ".dashboard { display:grid; grid-template-columns:repeat(auto-
fit,minmax(300px,1fr)); gap:30px;} ";
}
```



```
html += ".sensor-card { background:#f9f9f9; border-radius:15px; padding:25px; box-shadow:0  
10px 30px rgba(0,0,0,0.1);} ";  
html += ".sensor-value { font-size:2.5em; font-weight:bold; text-align:center;} ";  
html += ".sensor-bar { width:100%; height:8px; background:#eee; border-radius:4px;  
overflow:hidden; margin-top:10px;} ";  
html += ".sensor-fill { height:100%; transition:width 0.5s ease;} ";  
html += ".pot-fill { background:linear-gradient(90deg,#ff6b6b,#ee5a52);} ";  
html += ".ldr-fill { background:linear-gradient(90deg,#feca57,#ff9ff3);} ";  
html += "</style>";  
html += "</head>";
```

```
// ----- HTML Content -----
```

```
html += "<body>";  
html += "<div class='container'>";  
html += "<div class='header'><h1>ESP32 Sensor Dashboard</h1><p>Real-time monitoring of POT  
and LDR sensors</p></div>";
```

```
// Potentiometer card  
html += "<div class='dashboard'>";  
html += "<div class='sensor-card'>";  
html += "<h3>Potentiometer (POT)</h3>";  
html += "<div id='potValue' class='sensor-value'>0</div>";  
html += "<div class='sensor-bar'><div id='potBar' class='sensor-fill pot-fill'  
style='width:0%'></div></div>";  
html += "<p style='text-align:center;color:#666;font-size:0.9em;'>GPIO36 - Range: 0-  
4095</p>";  
html += "</div>";
```

```
// LDR card
html += "<div class='sensor-card'>";
html += "<h3>Light Sensor (LDR)</h3>";
html += "<div id='ldrValue' class='sensor-value'>0</div>";
html += "<div class='sensor-bar'><div id='ldrBar' class='sensor-fill ldr-fill'
style='width:0%'></div></div>";
html += "<p style='text-align:center;color:#666;font-size:0.9em;'>GPIO39 - Range: 0-
4095</p>";
html += "</div>";
html += "</div>";

// Last update time and auto-refresh option
html += "<div style='text-align:center;margin-top:20px;'>";
html += "<label><input type='checkbox' id='autoRefresh' checked> Auto-refresh every
20s</label>";
html += "<p id='lastUpdate' style='color:#888;'>Last updated: Never</p>";
html += "</div>";
```

```
// ----- JavaScript for Live Updates -----
```

```
html += "<script>";
html += "function fetchSensorData(){";
html += "fetch('/data').then(r=>r.json()).then(data=>{";
html += "document.getElementById('potValue').textContent=data.pot;";
html += "document.getElementById('ldrValue').textContent=data.ldr;";
html += "document.getElementById('potBar').style.width=(data.pot/4095*100)+'%';";
html += "document.getElementById('ldrBar').style.width=(data.ldr/4095*100)+'%';";
html += "document.getElementById('lastUpdate').textContent='Last updated: '+new
Date().toLocaleTimeString();";
html += "});}";
html += "setInterval(fetchSensorData,20000);"; // Auto-refresh every 20s
html += "fetchSensorData();"; // Initial load
html += "</script>";

html += "</body></html>";

// Send webpage to client
server.send(200, "text/html", html);
}
```

```
void handleData()  
{  
    // Send current sensor values to webpage  
    String json = "{\"pot\":\"" + String(potValue) + "\",\"ldr\":\"" + String(ldrValue) + "\"}";  
    server.send(200, "application/json", json);  
}
```

```
void handleThingSpeakData()
{
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;
        // Construct ThingSpeak API URL
        String url = "http://api.thingspeak.com/channels/" + channelID +
            "/feeds/last.json?api_key=" + readAPIKey;

        http.begin(url);
        int httpResponseCode = http.GET();

        if (httpResponseCode > 0) {
            String payload = http.getString();

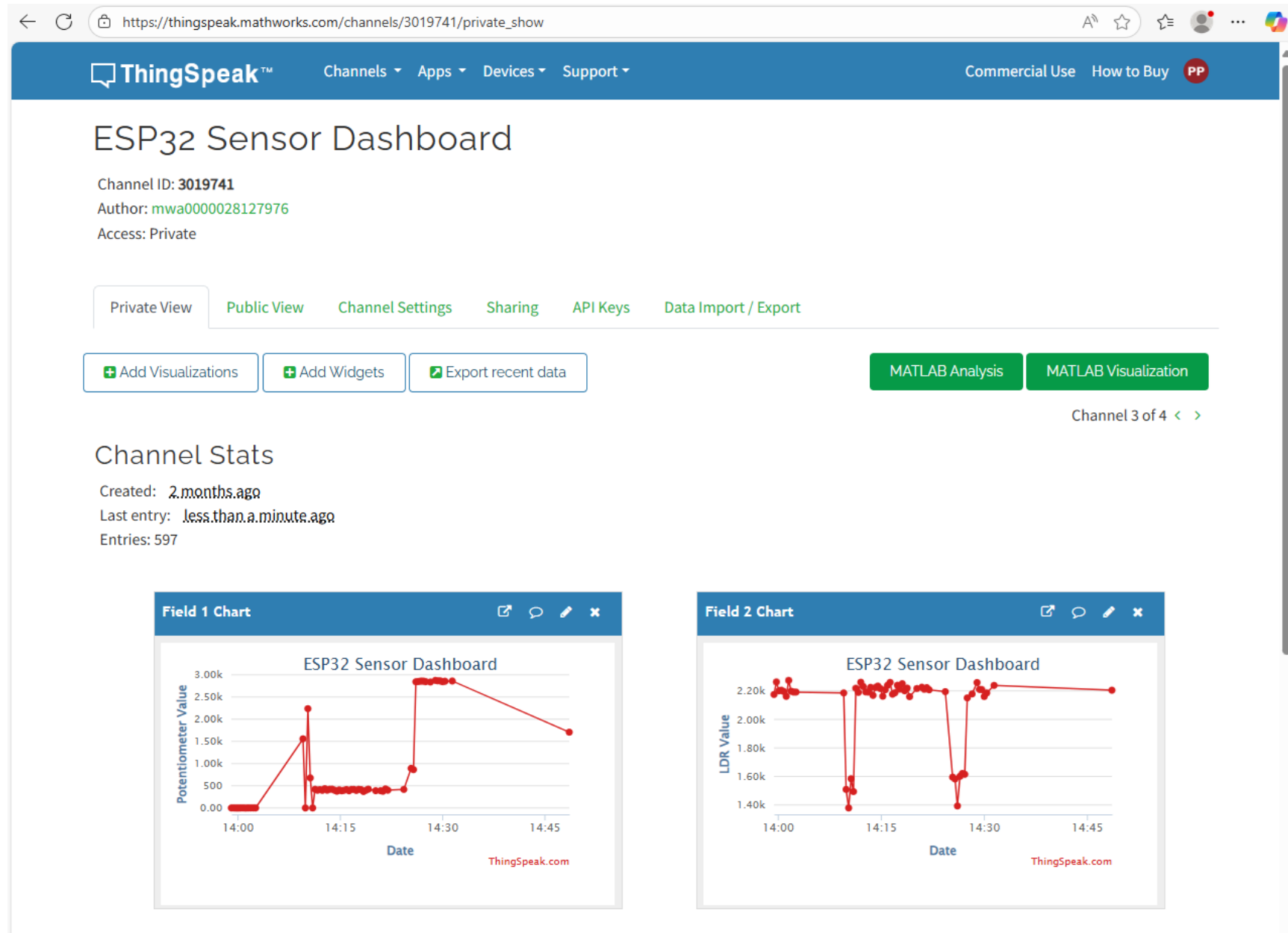
            // Manually extract field1 and field2 values from JSON
            int f1pos = payload.indexOf("\"field1\":\") + 10;
            int f1end = payload.indexOf("\",", f1pos);
            String field1 = payload.substring(f1pos, f1end);

            int f2pos = payload.indexOf("\"field2\":\") + 10;
            int f2end = payload.indexOf("\",", f2pos);
            String field2 = payload.substring(f2pos, f2end);
        }
    }
}
```

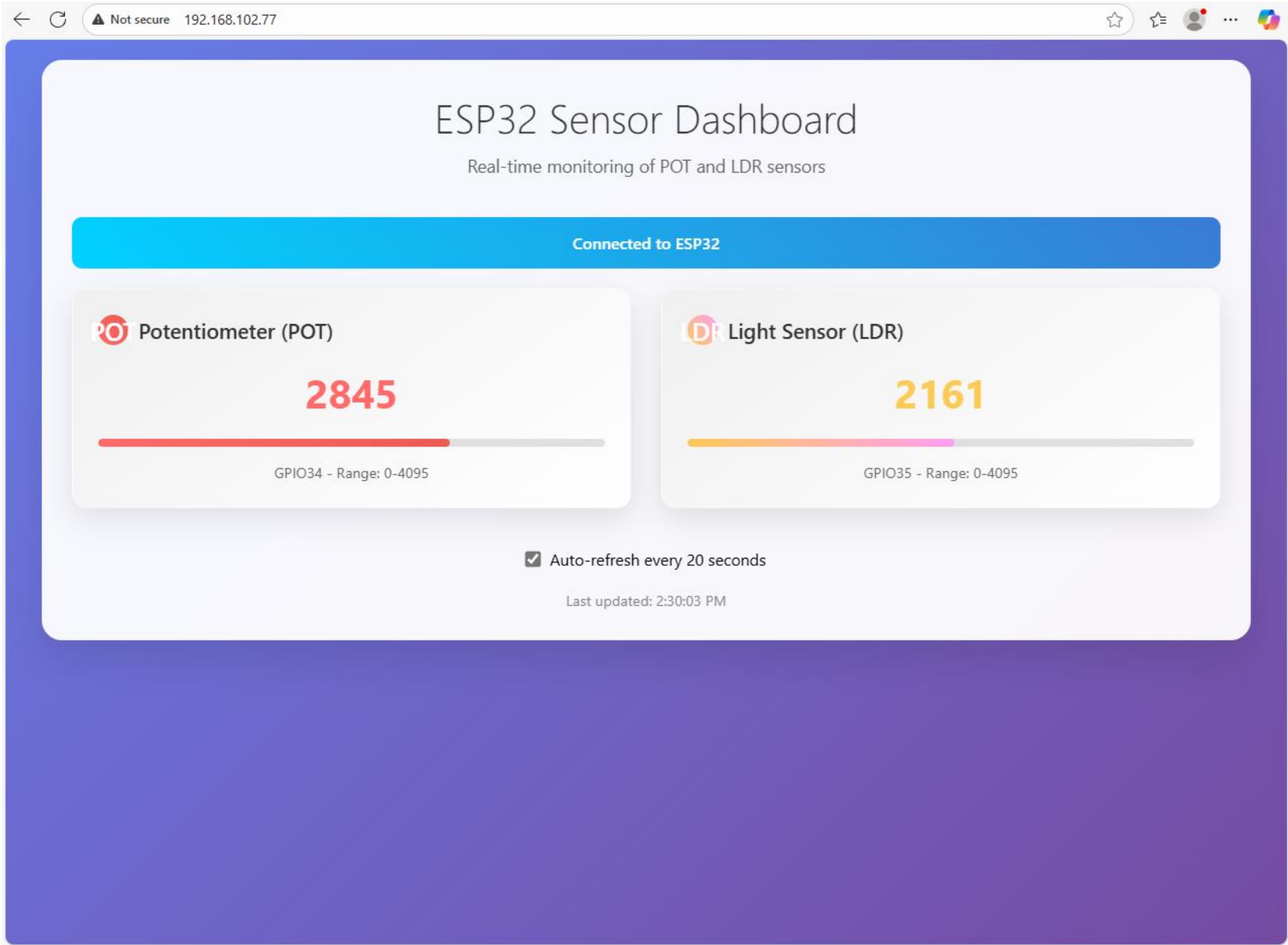
```
// Replace nulls with zero
    if (field1 == "null" || field1.length() == 0) field1 = "0";
    if (field2 == "null" || field2.length() == 0) field2 = "0";

    // Return JSON data to web browser
    String json = "{\"pot\":\"" + field1 + "\",\"ldr\":\"" + field2 + "\"}";
    server.send(200, "application/json", json);
} else {
    // Error case
    String json = "{\"pot\":0,\"ldr\":0}";
    server.send(200, "application/json", json);
}
http.end();
} else {
    // WiFi disconnected
    String json = "{\"pot\":0,\"ldr\":0}";
    server.send(200, "application/json", json);
}
}
```

Output - ThingSpeak Dashboard



Output - Web Dashboard



Thingspeak Web Integration with Sensor Monitor and Actuator Control

```
#include <WiFi.h>
#include <WebServer.h>
#include <HTTPClient.h>

// WiFi credentials
const char* ssid = "SSID";
const char* password = "Password";

// ThingSpeak configuration
const char* thingspeakHost = "api.thingspeak.com";
const String writeAPIKey = "CR9CQ5QZV7Q3N18T";
const String readAPIKey = "NJAPFN1Q80JFCS6T";
const String channelID = "3019740";

// Sensor pins
const int potPin = 36;      // POT sensor on GPIO36
const int ldrPin = 39;      // LDR sensor on GPIO39

// LED pins
const int LED1 = 33;        // LED1 on GPIO33
const int LED2 = 4;         // LED2 on GPIO4
```

```
// Web server
WebServer server(80);

// Global variables for sensor values
int potValue = 0;
int ldrValue = 0;
unsigned long lastThingSpeakUpdate = 0;
const unsigned long thingSpeakInterval = 20000; // Update every 20 seconds (ThingSpeak limit)

// LED states for status tracking
bool led1State = false;
bool led2State = false;

void setup() {
    Serial.begin(115200);

    // Initialize sensor pins
    pinMode(potPin, INPUT);
    pinMode(ldrPin, INPUT);
    // Initialize LED pins
    pinMode(LED1, OUTPUT);
    pinMode(LED2, OUTPUT);
    digitalWrite(LED1, LOW);
    digitalWrite(LED2, LOW);
```

```
// Connect to WiFi
WiFi.begin(ssid, password);
Serial.print("Connecting to WiFi");
while (WiFi.status() != WL_CONNECTED)
{
    delay(500);
    Serial.print(".");
}
Serial.println();
Serial.print("Connected to WiFi. IP address: ");
Serial.println(WiFi.localIP());

// Setup web server routes
server.on("/", handleRoot);
server.on("/thingspeak-data", handleThingSpeakData);
server.on("/led-control", handleLEDControl);

server.begin();
Serial.println("Web server started");
}
```

```
void loop()
{
    // Read sensor values
    potValue = analogRead(potPin);
    ldrValue = analogRead(ldrPin);

    // Send data to ThingSpeak every 20 seconds
    if (millis() - lastThingSpeakUpdate > thingSpeakInterval)
    {
        sendToThingSpeak();
        lastThingSpeakUpdate = millis();
    }

    // Handle web server requests
    server.handleClient();

    delay(100);
}
```

```
oid sendToThingSpeak()  
{  
    if (WiFi.status() == WL_CONNECTED)  
    {  
        HTTPClient http;  
        http.begin("http://api.thingspeak.com/update");  
        http.addHeader("Content-Type", "application/x-www-form-urlencoded");  
  
        String postData = "api_key=" + writeAPIKey + "&field1=" + String(potValue) + "&field2=" +  
String(ldrValue);  
  
        int httpResponseCode = http.POST(postData);  
  
        if (httpResponseCode > 0) {  
            String response = http.getString();  
            Serial.println("ThingSpeak Response: " + response);  
        } else {  
            Serial.println("Error sending to ThingSpeak: " + String(httpResponseCode));  
        }  
  
        http.end();  
    }  
}
```

```
void handleLEDControl() {
    String action = server.arg("action");
    String led = server.arg("led");
    if (led == "1") {
        if (action == "on") {
            digitalWrite(LED1, HIGH);
            led1State = true;
        } else if (action == "off") {
            digitalWrite(LED1, LOW);
            led1State = false;
        }
    } else if (led == "2") {
        if (action == "on") {
            digitalWrite(LED2, HIGH);
            led2State = true;
        } else if (action == "off") {
            digitalWrite(LED2, LOW);
            led2State = false;
        }
    }
    String json = "{\"led1\": " + String(led1State ? "true" : "false") + ", \"led2\": " +
String(led2State ? "true" : "false") + "}";
    server.send(200, "application/json", json);
}
```

```
void handleRoot()  
{  
    String html = "<!DOCTYPE html>";  
    html += "<html lang=\"en\">";  
    html += "<head>";  
    html += "<meta charset=\"UTF-8\">";  
    html += "<meta name=\"viewport\" content=\"width=device-width, initial-scale=1.0\">";  
    html += "<title>ESP32 Smart Home Dashboard</title>";  
    html += "<style>";  
    html += "* { margin: 0; padding: 0; box-sizing: border-box; }";  
    html += "body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; background: linear-gradient(135deg, #667eea 0%, #764ba2 100%); min-height: 100vh; padding: 20px; }";  
    html += ".container { max-width: 1400px; margin: 0 auto; background: rgba(255, 255, 255, 0.95); border-radius: 20px; padding: 30px; box-shadow: 0 20px 40px rgba(0, 0, 0, 0.1); backdrop-filter: blur(10px); }";  
    html += ".header { text-align: center; margin-bottom: 40px; }";  
    html += ".header h1 { color: #333; font-size: 2.5em; margin-bottom: 10px; font-weight: 300; }";  
    html += ".header p { color: #666; font-size: 1.1em; }";  
    html += ".device-info { background: linear-gradient(45deg, #3498db, #2980b9); color: white; padding: 15px; border-radius: 10px; margin-bottom: 30px; text-align: center; }";  
    html += ".main-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 30px; margin-bottom: 30px; }";  
}
```



```
html += ".sensor-section, .led-section { background: rgba(255, 255, 255, 0.5); border-  
radius: 15px; padding: 25px; }";  
html += ".section-title { font-size: 1.5em; font-weight: 600; color: #2c3e50; margin-bottom:  
20px; text-align: center; }";  
html += ".sensor-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 20px; }";  
html += ".sensor-card { background: linear-gradient(145deg, #f0f0f0, #ffffff); border-  
radius: 15px; padding: 20px; box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1); transition: transform  
0.3s ease; }";  
html += ".sensor-card:hover { transform: translateY(-5px); }";  
html += ".sensor-title { font-size: 1.2em; font-weight: 600; color: #333; margin-bottom:  
15px; display: flex; align-items: center; }";  
html += ".sensor-icon { width: 25px; height: 25px; margin-right: 10px; border-radius: 50%;  
display: flex; align-items: center; justify-content: center; font-size: 0.9em; color: white;  
}";  
html += ".pot-icon { background: linear-gradient(45deg, #ff6b6b, #ee5a52); }";  
html += ".ldr-icon { background: linear-gradient(45deg, #fecaa57, #ff9ff3); }";  
html += ".sensor-value { font-size: 2em; font-weight: bold; margin: 15px 0; text-align:  
center; }";  
html += ".pot-value { color: #ff6b6b; }";  
html += ".ldr-value { color: #fecaa57; }";  
html += ".sensor-bar { width: 100%; height: 6px; background: #e0e0e0; border-radius: 3px;  
overflow: hidden; margin: 10px 0; }";
```

```
html += ".sensor-fill { height: 100%; border-radius: 3px; transition: width 0.5s ease; }";
html += ".pot-fill { background: linear-gradient(90deg, #ff6b6b, #ee5a52); }";
html += ".ldr-fill { background: linear-gradient(90deg, #feca57, #ff9ff3); }";
html += ".led-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 20px; }";
html += ".led-card { background: linear-gradient(145deg, #f8f9fa, #ffffff); border-radius: 15px; padding: 20px; box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1); transition: transform 0.3s ease; }";
html += ".led-card:hover { transform: translateY(-5px); }";
html += ".led-header { display: flex; align-items: center; justify-content: space-between; margin-bottom: 15px; }";
html += ".led-title { font-size: 1.2em; font-weight: 600; color: #2c3e50; }";
html += ".led-status { display: flex; align-items: center; gap: 6px; }";
html += ".status-indicator { width: 10px; height: 10px; border-radius: 50%; transition: all 0.3s ease; }";
html += ".status-on { background: #27ae60; box-shadow: 0 0 10px rgba(39, 174, 96, 0.6); animation: pulse-green 2s infinite; }";
html += ".status-off { background: #e74c3c; }";
html += ".status-text { font-weight: 500; font-size: 0.8em; }";
html += ".led-visual { text-align: center; margin: 15px 0; }";
html += ".led-bulb { width: 60px; height: 60px; border-radius: 50%; margin: 0 auto 10px; transition: all 0.3s ease; display: flex; align-items: center; justify-content: center; font-size: 1.5em; }";
```

```
html += ".led1-on { background: linear-gradient(45deg, #ff6b6b, #ee5a52); box-shadow: 0 0 20px rgba(255, 107, 107, 0.6); color: white; animation: glow-red 3s infinite; }";
html += ".led1-off { background: linear-gradient(45deg, #bdc3c7, #95a5a6); color: #7f8c8d; }";
html += ".led2-on { background: linear-gradient(45deg, #ff6b6b, #ee5a52); box-shadow: 0 0 20px rgba(255, 107, 107, 0.6); color: white; animation: glow-red 3s infinite; }";
html += ".led2-off { background: linear-gradient(45deg, #bdc3c7, #95a5a6); color: #7f8c8d; }";
html += ".control-buttons { display: flex; gap: 8px; justify-content: center; }";
html += ".btn { padding: 8px 16px; border: none; border-radius: 20px; font-weight: 600; font-size: 12px; cursor: pointer; transition: all 0.3s ease; }";
html += ".btn-on { background: linear-gradient(45deg, #27ae60, #2ecc71); color: white; }";
html += ".btn-off { background: linear-gradient(45deg, #e74c3c, #c0392b); color: white; }";
html += ".btn:hover { transform: translateY(-2px); }";
html += ".data-source { text-align: center; margin-bottom: 20px; }";
html += ".status { text-align: center; padding: 10px; border-radius: 8px; margin-bottom: 15px; font-weight: 500; font-size: 0.9em; }";
html += ".status.connected { background: linear-gradient(45deg, #00d2ff, #3a7bd5); color: white; }";
html += ".status.loading { background: linear-gradient(45deg, #fecaa5, #ff9ff3); color: white; }";
html += ".last-update { text-align: center; color: #888; font-size: 0.8em; margin-top: 15px; }";
```

```
html += ".auto-refresh { display: flex; align-items: center; justify-content: center;
margin-top: 15px; gap: 8px; font-size: 0.9em; }";
html += ".auto-refresh input[type=\"checkbox\"] { transform: scale(1.1); }";
html += "@keyframes pulse-green { 0%, 100% { opacity: 1; } 50% { opacity: 0.5; } }";
html += "@keyframes glow-red { 0%, 100% { box-shadow: 0 0 20px rgba(255, 107, 107, 0.6); }
50% { box-shadow: 0 0 30px rgba(255, 107, 107, 0.9); } }";
html += "@keyframes glow-teal { 0%, 100% { box-shadow: 0 0 20px rgba(78, 205, 196, 0.6); }
50% { box-shadow: 0 0 30px rgba(78, 205, 196, 0.9); } }";
html += "@media (max-width: 768px) { .main-grid { grid-template-columns: 1fr; } .sensor-
grid, .led-grid { grid-template-columns: 1fr; } .container { padding: 20px; margin: 10px; }
}";
html += "</style>";
html += "</head>";
html += "<body>";
html += "<div class=\"container\">";
html += "<div class=\"header\">";
html += "<h1>ESP32 Smart Home Dashboard</h1>";
html += "<p>Sensor Monitoring & LED Control System</p>";
html += "</div>";
html += "<div class=\"device-info\">";
html += "<p><strong>Device:</strong> ESP32 | <strong>IP:</strong> " +
WiFi.localIP().toString() + " | <strong>Status:</strong> Online</p>";
html += "</div>";
html += "<div class=\"main-grid\">";
```

```
// Sensor Section
html += "<div class=\"sensor-section\">";
html += "<div class=\"section-title\">Sensor Data (ThingSpeak)</div>";
html += "<div class=\"data-source\">";
html += "<div class=\"status connected\" id=\"status\">ThingSpeak Cloud Data</div>";
html += "</div>";
html += "<div class=\"sensor-grid\">";
html += "<div class=\"sensor-card\">";
html += "<div class=\"sensor-title\">";
html += "<div class=\"sensor-icon pot-icon\">POT</div>";
html += "Potentiometer";
html += "</div>";
html += "<div class=\"sensor-value pot-value\" id=\"potValue\">0</div>";
html += "<div class=\"sensor-bar\">";
html += "<div class=\"sensor-fill pot-fill\" id=\"potBar\" style=\"width: 0%\"></div>";
html += "</div>";
html += "<div style=\"text-align: center; color: #666; font-size: 0.8em;\">GPIO34</div>";
html += "</div>";
html += "<div class=\"sensor-card\">";
html += "<div class=\"sensor-title\">";
html += "<div class=\"sensor-icon ldr-icon\">LDR</div>";
```

```
html += "Light Sensor";
html += "</div>";
html += "<div class=\"sensor-value ldr-value\" id=\"ldrValue\">0</div>";
html += "<div class=\"sensor-bar\">";
html += "<div class=\"sensor-fill ldr-fill\" id=\"ldrBar\" style=\"width: 0%\"></div>";
html += "</div>";
html += "<div style=\"text-align: center; color: #666; font-size: 0.8em;\">GPIO35</div>";
html += "</div>";
html += "</div>";
html += "<div class=\"auto-refresh\">";
html += "<input type=\"checkbox\" id=\"autoRefresh\" checked>";
html += "<label for=\"autoRefresh\">Auto-refresh every 20 seconds</label>";
html += "</div>";
html += "</div>";
// LED Control Section
html += "<div class=\"led-section\">";
html += "<div class=\"section-title\">LED Control</div>";
html += "<div class=\"led-grid\">";
html += "<div class=\"led-card\">";
html += "<div class=\"led-header\">";
html += "<div class=\"led-title\">LED 1</div>";
html += "<div class=\"led-status\">";
html += "<div class=\"status-indicator \" + String(led1State ? \"status-on\" : \"status-off\") +";
html += "</div>";
```

```
html += "<span class=\"status-text\">" + String(led1State ? "ON" : "OFF") + "</span>";
html += "</div>";
html += "</div>";
html += "<div class=\"led-visual\">";
html += "<div class=\"led-bulb \" + String(led1State ? \"led1-on\" : \"led1-off\") +
\">💡</div>";
html += "</div>";
html += "<div style=\"text-align: center; color: #666; font-size: 0.8em; margin-bottom:
10px;\">GPIO33</div>";
html += "<div class=\"control-buttons\">";
html += "<button class=\"btn btn-on\" onclick=\"controlled(1, 'on')\">ON</button>";
html += "<button class=\"btn btn-off\" onclick=\"controlled(1, 'off')\">OFF</button>";
html += "</div>";
html += "</div>";
html += "<div class=\"led-card\">";
html += "<div class=\"led-header\">";
html += "<div class=\"led-title\">LED 2</div>";
html += "<div class=\"led-status\">";
html += "<div class=\"status-indicator \" + String(led2State ? \"status-on\" : \"status-off\") +
\"></div>";
html += "<span class=\"status-text\">" + String(led2State ? "ON" : "OFF") + "</span>";
html += "</div>";
html += "</div>";
```

```
html += "<div class=\"led-visual\">";
html += "<div class=\"led-bulb \" + String(led2State ? \"led2-on\" : \"led2-off\") +
\">💡</div>";
html += "</div>";
html += "<div style=\"text-align: center; color: #666; font-size: 0.8em; margin-bottom:
10px;\">GPIO4</div>";
html += "<div class=\"control-buttons\">";
html += "<button class=\"btn btn-on\" onclick=\"controlLED(2, 'on')\">ON</button>";
html += "<button class=\"btn btn-off\" onclick=\"controlLED(2, 'off')\">OFF</button>";
html += "</div>";
html += "</div>";
html += "</div>";
html += "</div>";
html += "</div>";

html += "<div class=\"last-update\" id=\"lastUpdate\">Last updated: Never</div>";

html += "</div>";
```



```
html += "<script>";
html += "var autoRefreshInterval;";
html += "function fetchSensorData() {";
html += "var url = '/thingspeak-data';";
html += "var status = document.getElementById('status');";
html += "status.textContent = 'Fetching from ThingSpeak Cloud';";
html += "status.className = 'status loading';";
html += "fetch(url)";
html += ".then(function(response) { return response.json(); })";
html += ".then(function(data) {";
html += "updateSensorDisplay(data.pot, data.ldr);";
html += "updateLastUpdate();";
html += "status.textContent = 'Data from ThingSpeak Cloud';";
html += "status.className = 'status connected';";
html += "});";
html += ".catch(function(error) {";
html += "console.error('Error fetching data:', error);";
html += "status.textContent = 'Connection Error';";
html += "status.className = 'status loading';";
html += "});";
html += "}";
```

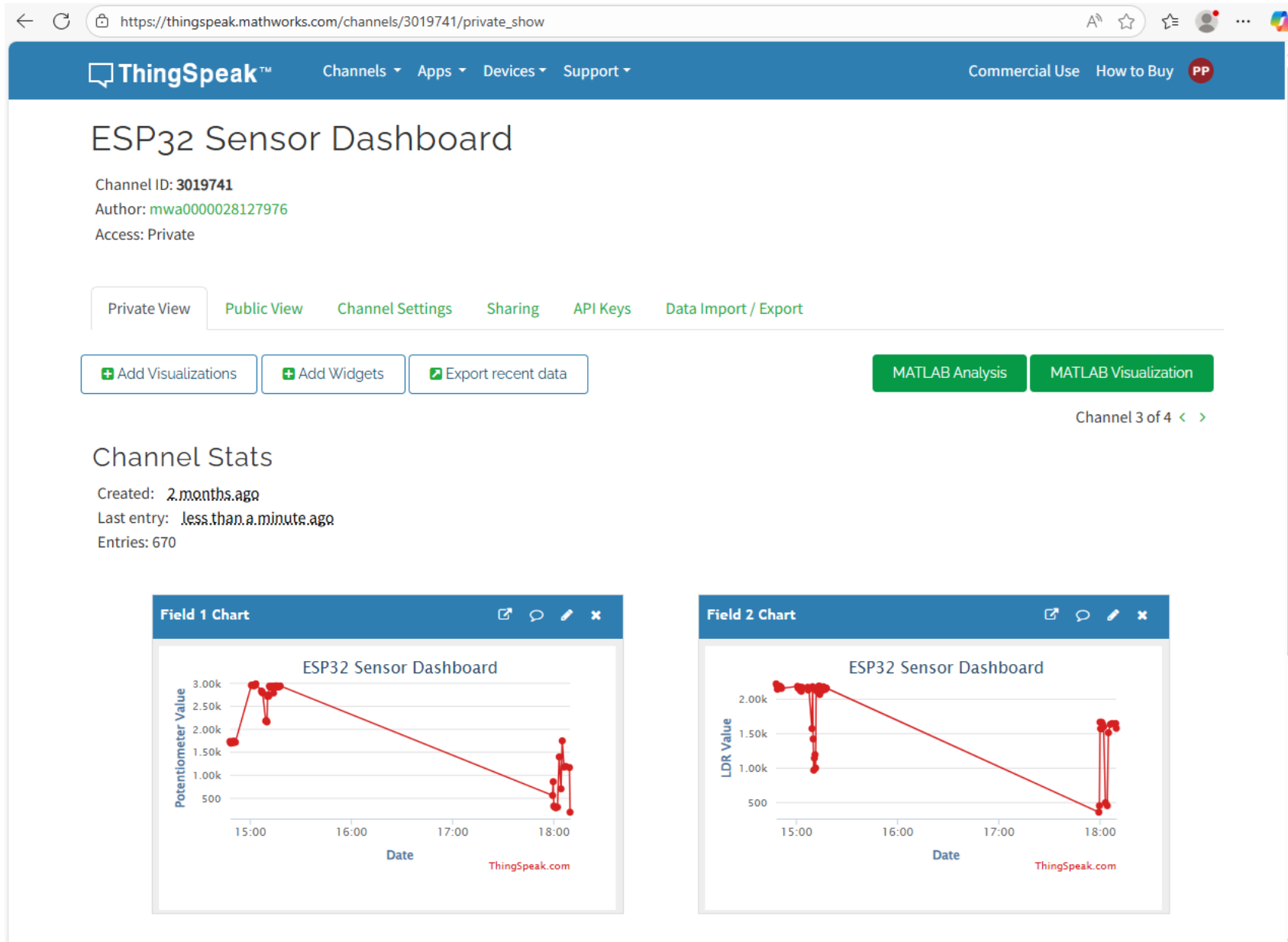
```
html += "function updateSensorDisplay(potVal, ldrVal) {";  
html += "document.getElementById('potValue').textContent = potVal;";  
html += "document.getElementById('ldrValue').textContent = ldrVal;";  
html += "var potPercent = (potVal / 4095) * 100;";  
html += "var ldrPercent = (ldrVal / 4095) * 100;";  
html += "document.getElementById('potBar').style.width = potPercent + '%';";  
html += "document.getElementById('ldrBar').style.width = ldrPercent + '%';";  
html += "}";  
html += "function updateLastUpdate() {";  
html += "var now = new Date();";  
html += "var timeString = now.toLocaleTimeString();";  
html += "document.getElementById('lastUpdate').textContent = 'Last updated: ' +  
timeString;";  
html += "}";  
html += "function controlLED(ledNum, action) {";  
html += "fetch('/led-control?led=' + ledNum + '&action=' + action)";  
html += ".then(function(response) { return response.json(); })";  
html += ".then(function(data) {";  
html += "setTimeout(function() { location.reload(); }, 500);";  
html += "})";  
html += ".catch(function(error) {";  
html += "console.error('Error controlling LED:', error);";  
html += "});";  
html += "}";
```

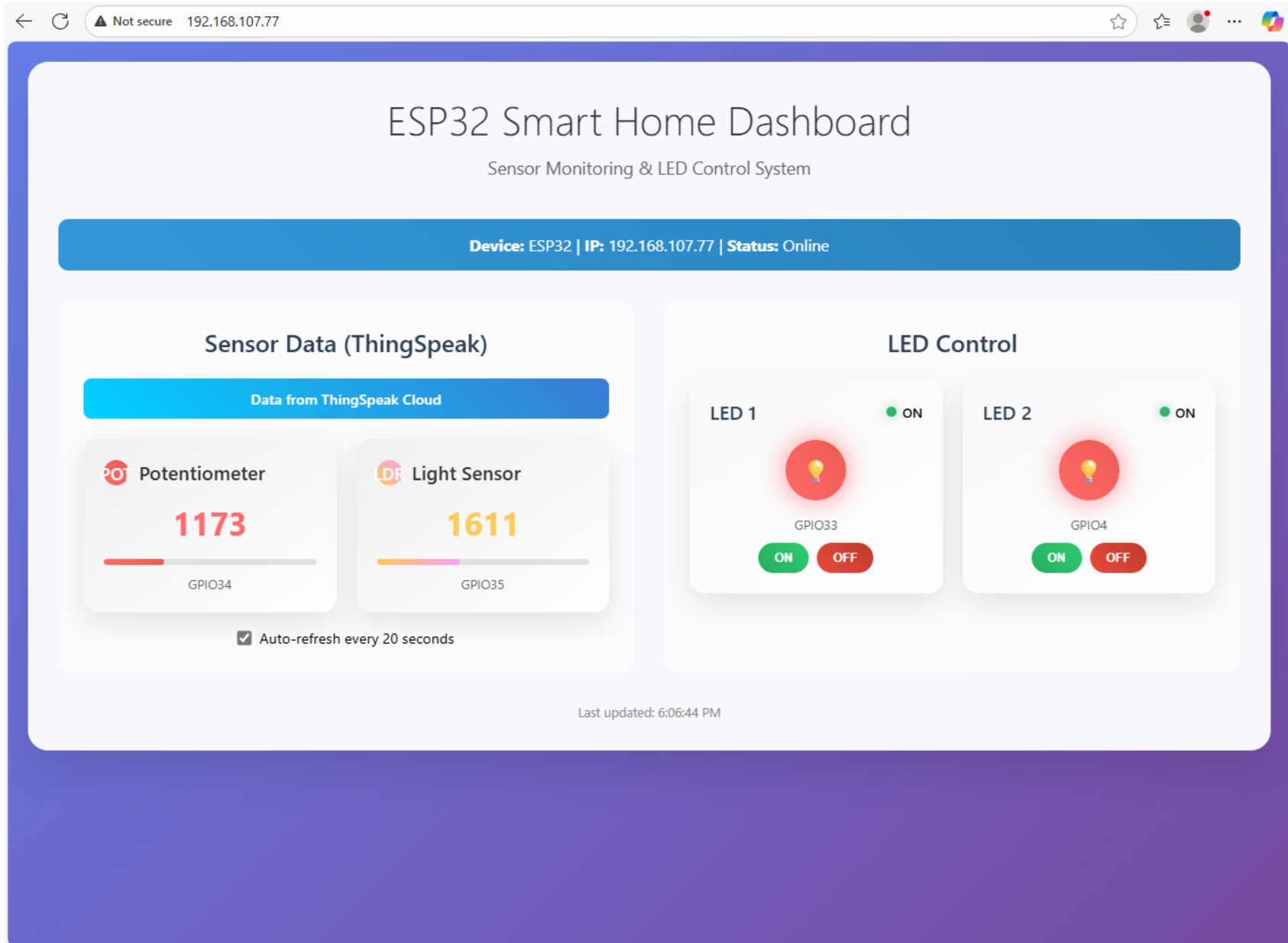
```
html += "function setupAutoRefresh() {";  
html += "var checkbox = document.getElementById('autoRefresh');";  
html += "function startAutoRefresh() {";  
html += "if (autoRefreshInterval) { clearInterval(autoRefreshInterval); }";  
html += "autoRefreshInterval = setInterval(fetchSensorData, 20000);";  
html += "}";  
html += "function stopAutoRefresh() {";  
html += "if (autoRefreshInterval) { clearInterval(autoRefreshInterval); autoRefreshInterval  
= null; }";  
html += "}";  
html += "checkbox.addEventListener('change', function() {";  
html += "if (this.checked) { startAutoRefresh(); } else { stopAutoRefresh(); }";  
html += "});";  
html += "if (checkbox.checked) { startAutoRefresh(); }";  
html += "}";  
html += "fetchSensorData();";  
html += "setupAutoRefresh();";  
html += "</script>";  
html += "</body>";  
html += "</html>";  
  
server.send(200, "text/html", html);  
}
```

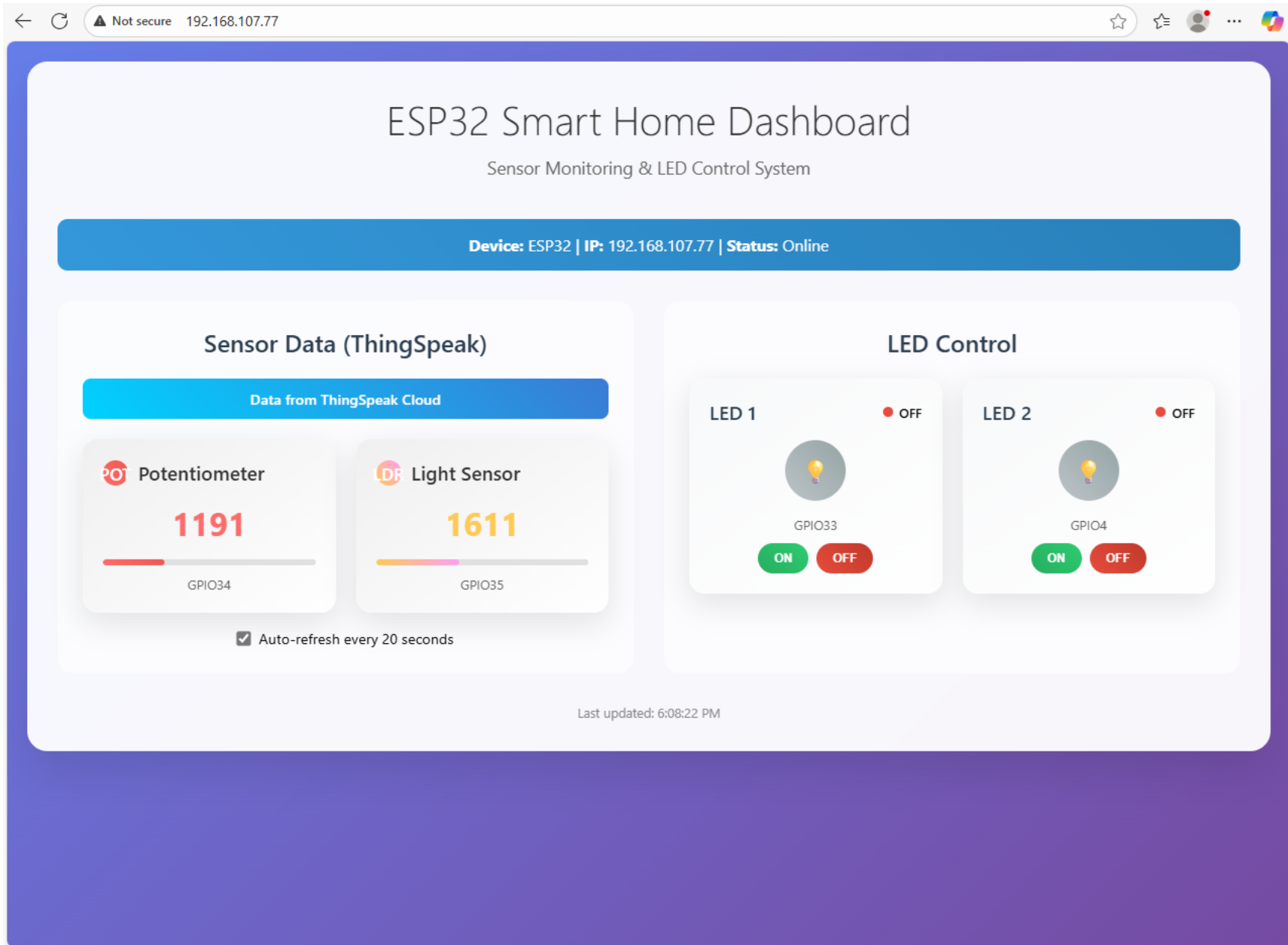
```
void handleThingSpeakData() {  
    if (WiFi.status() == WL_CONNECTED) {  
        HTTPClient http;  
        String url = "http://api.thingspeak.com/channels/" + channelID +  
"/feeds/last.json?api_key=" + readAPIKey;  
  
        http.begin(url);  
        int httpResponseCode = http.GET();  
  
        if (httpResponseCode > 0) {  
            String payload = http.getString();  
  
            // Parse JSON manually (without ArduinoJson)  
            int field1Pos = payload.indexOf("\"field1\":") + 10;  
            int field1End = payload.indexOf("\"", field1Pos);  
            String field1Value = payload.substring(field1Pos, field1End);  
            int field2Pos = payload.indexOf("\"field2\":") + 10;  
            int field2End = payload.indexOf("\"", field2Pos);  
            String field2Value = payload.substring(field2Pos, field2End);  
  
            // Handle null values  
            if (field1Value == "null" || field1Value.length() == 0) field1Value = "0";  
            if (field2Value == "null" || field2Value.length() == 0) field2Value = "0";  
        }  
    }  
}
```

```
String json = "{\"pot\":\"" + field1Value + "\",\"ldr\":\"" + field2Value + "\"}";
server.send(200, "application/json", json);
} else {
String json = "{\"pot\":0,\"ldr\":0}";
server.send(200, "application/json", json);
}

http.end();
} else {
String json = "{\"pot\":0,\"ldr\":0}";
server.send(200, "application/json", json);
}
}
```







Thank You